# CPSC 304 Project Cover Page

Milestone #: 2

Date: July 22, 2025

Group Number: 21

| Name | Student Number | CS Alias (Userid) | Preferred E-mail Address |
|---|---|---|---|
| Arshia Zargaran | 72033004 | x4e7u | arshia.zargaran@gmail.com |
| Katelyn Chun | 19372317 | b4v6q | chunkatelyn@gmail.com |
| Amishi Arora | 62983465 | k5w4p | amishia859@gmail.com |

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

## Summary of Project:

Our project is a recipe management system. Users can upload, favourite, and review various recipes. Recipes have ingredients, equipment, steps, and different tags (like dietary restrictions and cuisine types). Users can also sign up for courses taught by professional chefs that demo some of the recipes.

## ER Diagram Changes:

- Added total participation constraint and key constraint for review in the reviews relation because a review is related to exactly one recipe
- Added rank attribute to the professional entity (for a meaningful functional dependency)
- Added cost as a descriptive attribute for contains relationship (for a meaningful functional dependency)
- Got rid of some of the recipe attributes (protein, fat, carbs, calories). These did not add much to our application and also did not give any functional dependencies
- Added a name attribute to course
- Got rid of verified for professional attribute, because all professionals should be verified by default (also did not present any functional dependencies)
- Got rid of createdBy attribute in recipe entity because that's a foreign key
- Got rid of createdBy and recipeId in the review entity because those are foreign keys. Replaced with reviewID instead as a key.
- Added description for step attribute
- Minor adjustments to attribute names to be consistent (eg. ID to uID)

## Schema:

recipe(rID, **uID**, title, desc, serving)
user(uID, email, uName)
favourites(**uID**, **rID**)
Professional(**uID**, YOE, speciality, rank)
ingredient(iName, type)
hasTag(**tName**, **rID)**

equipment(eName, whereToBuy)
registers(**uID**, **cID**, date)
demosRecipe(**rID**, **cID**)
contains(**rID**, **iName**, amount, price)
tag(tName, type)
step(**rID**, stepNumber, image, description)
requiresEq(**rID**, **eName**)
course(cID, **proID**, cName, duration, difficulty, price)
review(reviewID, **rID**, **uID**, rating, comment)

## Functional Dependencies:

Candidate / Primary Keys:
recipeId, userID -> title, description, servings
uID -> email, uName
uID, cID -> date
uID -> YOE, speciality, rank
iName -> type
eName -> whereTobuy
rID, iName -> amount, price
tName -> type
rID, stepNumber -> image, description
cID -> proID, cName, duration, difficulty, price
reviewID -> rID, uID, rating, comment

Other:
YOE -> Rank
ingredientID, amount -> cost
prolD, duration, difficulty -> price

## Normalization in BCNF:

recipe(rID, **uID**, title, desc, serving)
user(uID, email, uName)
favourites(**uID**, **rID**)
ingredient(iName, type)

hasTag(**tName**, **rID)**
equipment(eName, whereToBuy)
registers(**uID**, **cID**, date)
demosRecipe(**rID**, **cID**)
tag(tName, type)
step(**rID**, stepNumber, image, description)
requiresEq(**rID**, **eName**)
review(reviewID, **rID**, **uID**, rating, comment)

professional(**uId**, YOE, speciality, rank) normalized into:
professionalOne(YOE, rank)
professionalTwo(**uId**, YOE, speciality)

contains(**rID**, **iName**, cost, amount) normalized into:
containsOne(**iName**, amount, cost)
containsTwo(**rID**, **iName**, amount)

courses(cID, cName, duration, difficulty, price, teacher) normalized into:
coursesOne(**uID**, duration, difficulty, price)
coursesTwo(cID, **teacherID,** cName, difficulty, duration)

## DDL Statements:

```
CREATE TABLE recipe (
        rID             INT,
        title           VARCHAR(50),
        description     TEXT,
        uID             INT     NOT NULL,
        servings        INT,
        PRIMARY KEY (rID),
        FOREIGN KEY (uID) REFERENCES user(uID) ON DELETE CASCADE);

CREATE TABLE user (
        uID         INT,
        email       VARCHAR(50),
```

```
    uName       VARCHAR(50),
    PRIMARY KEY (uID));

CREATE TABLE favourites (
    rID         INT,
    uID         INT,
    PRIMARY KEY (rID, uID),
    FOREIGN KEY (rID) REFERENCES recipe(rID) ON DELETE CASCADE,
    FOREIGN KEY (uID) REFERENCES user(uID) ON DELETE CASCADE);

CREATE TABLE ingredient (
    iName       VARCHAR(30),
    type        VARCHAR(30),
    PRIMARY KEY (iID));

CREATE TABLE hasTag (
    rID         INT,
    tName       VARCHAR(30),
    PRIMARY KEY (rID, tName),
    FOREIGN KEY (rID) REFERENCES recipe(rID) ON DELETE CASCADE,
    FOREIGN KEY (tName) REFERENCES tag(tName));

CREATE TABLE equipment (
    eName           VARCHAR(50),
    whereToBuy      VARCHAR(50),
    PRIMARY KEY (eName));

CREATE TABLE registers (
    uID         INT,
    cID         INT,
    date        DATE,
    PRIMARY KEY (uID, cID),
    FOREIGN KEY (uID) REFERENCES user(uID) ON DELETE CASCADE,
    FOREIGN KEY (cID) REFERENCES course(cID) ON DELETE CASCADE);
```

```
CREATE TABLE demosRecipe(
        rID             INT,
        cID             INT,
        PRIMARY KEY (rID, cID),
        FOREIGN KEY (rID) REFERENCES recipe(rID) ON DELETE CASCADE,
        FOREIGN KEY (cID) REFERENCES course(cID) ON DELETE CASCADE);

CREATE TABLE professionalOne (
        YOE             INT,
        rank            VARCHAR(10),
        PRIMARY KEY (YOE));

CREATE TABLE professionalTwo (
        uID             INT,
        YOE             INT,
        speciality      VARCHAR(30),
        PRIMARY KEY (uID),
        FOREIGN KEY (uID) REFERENCES user(uID) ON DELETE CASCADE);

CREATE TABLE containsOne (
        iName           VARCHAR(30),
        amount          VARCHAR(30),
        cost            NUMBER(5,2),
        PRIMARY KEY (iID, amount),
        FOREIGN KEY (iID) REFERENCES ingredient(iID));

CREATE TABLE containsTwo (
        iName           VARCHAR(30),
        rID             INT,
        amount          VARCHAR(30),
        PRIMARY KEY (iName, rID),
        FOREIGN KEY (iName) REFERENCES ingredient(iName),
        FOREIGN KEY (rID) REFERENCES recipe(rID));

CREATE TABLE courseOne (
```

```
    teacherID       INT     NOT NULL,
    duration        INT,
    difficulty      VARCHAR(10),
    price           NUMBER(5,2),
    PRIMARY KEY (teacherID, duration, difficulty),
    FOREIGN KEY (teacherID) REFERENCES professionalTwo(uID) ON DELETE
CASCADE);

CREATE TABLE courseTwo (
    cID             INT,
    cName           VARCHAR(30),
    teacherID       INT     NOT NULL,
    duration        INT,
    difficulty      VARCHAR(10),
    PRIMARY KEY (cID),
    FOREIGN KEY (teacherID) REFERENCES professionalTwo(uID) ON DELETE
CASCADE);

CREATE TABLE tag (
    tName       VARCHAR(50),
    type        VARCHAR(50),
    PRIMARY KEY (tName));

CREATE TABLE step (
    rID             INT,
    stepNumber      INT,
    image           VARCHAR(50),
    PRIMARY KEY (rID, stepNumber),
    FOREIGN KEY (rID) REFERENCES recipe(rID) ON DELETE CASCADE);

CREATE TABLE requires (
    rID         INT,
    eName       VARCHAR(50),
    PRIMARY KEY (rID, eName),
    FOREIGN KEY (rID) REFERENCES recipe(rID) ON DELETE CASCADE);
```

```
CREATE TABLE review (
        reviewID    INT,
        rID         INT NOT NULL,
        uID         INT NOT NULL,
        rate        INT,
        comment     TEXT,
        PRIMARY KEY (reviewID),
        FOREIGN KEY (rID) REFERENCES recipe(rID) ON DELETE CASCADE),
        FOREIGN KEY (uID) REFERENCES user(uID) ON DELETE CASCADE);
```

## Insert Statements:

```
INSERT INTO recipe (rID, title, description, uID, servings ) VALUES
(101, 'Chocolate Chip Cookies', 'Moist and soft chocolate chip cookies.', 101,
10),
(102, 'Mac and Cheese', 'Cheesey mac and cheese everyone will love.', 102, 4),
(3, 'Veggie Stir-fry', 'Quick and healthy stir-fried vegetables.', 102, 2),
(104, 'Spaghetti Bolognese', 'Classic Italian pasta with rich meat sauce', 103, 2),
(105, 'Chocolate Cake', 'Decadent chocolate cake', 103, 10);

INSERT INTO user (uID, email, uName) VALUES
(101, 'johnsmith@ubc.ca', 'John Smith'),
(102, 'maryjane@gmail.com', 'Mary Jane'),
(103, 'jpark@yahoo.ca', 'Jerry Park'),
(104, 'bobbuilder@ubc.ca', 'Bob Builder'),
(105, 'ssunnypatch@sfu.ca', 'Sarah Sunnypatch');

INSERT INTO favourites (rID, uID) VALUES
(101, 102),
(102, 101),
(103, 101),
(104, 102),
(105, 101);
```

```
INSERT INTO ingredient (iName, type) VALUES
('corn', 'vegetable'),
('macaroni', 'carbohydrate'),
('chicken', 'protein'),
(flour, 'baking'),
('pepper', 'seasoning');

INSERT INTO hasTag (rID, tName) VALUES
(104, 'Italian'),
(104, 'Pasta'),
(103, 'Vegetarian'),
(103, 'Quick'),
(101, 'Dessert');

INSERT INTO equipment (eName, whereToBuy) VALUES
('whisk', 'Ikea'),
('handmixer', 'Home Depot'),
('cast iron skillet', 'Ikea'),
('mandolin', 'Canadian Tire'),
('ladle', 'Ikea');

INSERT INTO registers(uID, cID, date) VALUES
(101, 101, '2025-07-01'),
(102, 101, '2025-07-01'),
(101, 102, '2025-07-02'),
(103, 101, '2025-07-02'),
(101, 103, '2025-07-03');

INSERT INTO demosRecipe(uID, cID) VALUES
(101, 101),
(105, 103),
(101, 102),
(105, 104),
(101, 104);
```

```
INSERT INTO professionalOne (YOE, rank) VALUES
(1, 'Junior'),
(2, 'Junior'),
(5, 'Mid'),
(6, 'Mid'),
(10, 'Senior');

INSERT INTO professionalTwo (uID, YOE, speciality) VALUES
(101, 5, 'Italian cuisine'),
(102, 10, 'Vegetarian recipes'),
(103, 1, 'Greek cuisine'),
(104, 2, 'Baker'),
(105, 6, 'Asian cuisine');

INSERT INTO containsOne (iName, amount, cost) VALUES
('flour', '30g', '2.11'),
('corn' '1 cup', '4.50'),
('macaroni', '2 cups', '5.00'),
('chicken', '3 drumsticks', '9.50'),
('pepper', '0.5 tsp', '1.00');

INSERT INTO containsTwo (rID, iName, amount) VALUES
(101, 'flour', '50g'),
(105, 'flour', '100g'),
(103, 'corn', '1 cup'),
(104, 'pepper', '1 tsp'),
(102, 'macaroni', '1.5 cups');

INSERT INTO courseOne (teacherID, duration, difficulty, price) VALUES
(101, 10, 'easy', 9.99),
(102, 20, 'hard', 99.99),
(103, 10, 'easy', 4.99),
(102, 5, 'medium', 44.99),
(105, 15, 'medium', 19.99);
```

```
INSERT INTO courseOne (cID, cName, teacherID, duration, difficulty) VALUES
(101, 'Desserts', 101, 10, 'easy'),
(102, 'Meals for Students', 103, 10, 'easy'),
(103, 'Everything Veggie', 102, 5, 'medium'),
(104, 'A Nice Meal for a Nice Date', 105, 15, 'medium'),
(105, 'Cook It and Lean It', 102, 20, 'hard');

INSERT INTO tag (tName, type) VALUES
('Italian', 'region'),
('Pasta', 'dish'),
('Vegetarian', 'diet'),
('Dessert', 'course'),
('Quick', 'time');

INSERT INTO step (rID, stepNumber, description , image) VALUES
(102, 1, 'prepare the cheese', NULL),
(102, 2, 'prepare the mac',NULL),
(102, 3, 'mix them', NULL),
(101, 1, 'melt the chocolate', NULL),
(101, 2, 'put in in the oven', NULL);

INSERT INTO requires (rID, eName) VALUES
(104, 'mandolin'),
(101, 'handmixer'),
(105, 'whisk'),
(104, 'cast iron skillet'),
(103, 'ladle');

INSERT INTO review (reviewID, rID, uID, rate, comment) VALUES
(101, 101, 102, 4, 'It was too sweet'),
(102, 102, 103, 8, 'Easy and delicious'),
(103, 103, 103, 4, 'It would be better with more salt'),
(104, 101, 104, 9, 'Reminded me of the cookies my grandma made me'),
(105, 104, 104, 10, 'I totally loved it');
```